Equilibrium Pressure Extractor: Technical Documentation

Pressure Tool

2026-05-11

Overview

The **Equilibrium Pressure Extractor** is an interactive web-based tool for analyzing time-series pressure and signal data from chemical equilibrium experiments. It provides features for:

- Loading and visualizing CSV data
- Defining regions of interest by dragging on the chart
- Computing statistics (mean, std, count) for each region
- Applying linear drift corrections to signal data
- Exporting corrected data in CSV or TSV format

This tool is designed for VLE (vapor-liquid equilibrium) measurements where signal drift over time must be corrected before computing equilibrium properties.

Architecture

Data Flow

```
CSV Input
    ↓
parseTimeValue() → parsedRows (raw x, y pairs)
    ↓
Normalize x: x_normalized = x - base (first x value)
    ↓
parsedOriginal (immutable reference)
```

```
    ↓
applyLinearCorrection()
    ↓
rawData (displayed data, corrected if enabled)
    ↓
Chart.js visualization + Region statistics
    ↓
Export (CSV/TSV)
```

Key Data Structures

originalData: Raw CSV rows (unmodified, for column selection)

parsedOriginal: Normalized time-series data (immutable baseline)

```
let parsedOriginal = [
  { x: 0,      y: 45.2 },  // first point (base subtracted)
  { x: 10.5,   y: 45.1 },
  { x: 21.0,   y: 44.8 },
  // ...
]
```

rawData: Currently displayed data (corrected if linear correction enabled)

```
let rawData = [
  { x: 0,      y: 45.2 },
  { x: 10.5,   y: 45.1 },
  { x: 21.0,   y: 44.8 },
]
```

regions: User-defined selection regions for statistics

```
let regions = [
  { x0: 5, x1: 15, color: 'hsla(47, 85%, 60%, 0.35)' },
  { x0: 30, x1: 40, color: 'hsla(94, 85%, 60%, 0.35)' },
]
```

correction: Linear correction parameters

```
let correction = {
  enabled: false,      // checkbox state
  slope: 0,            // m in y = mx + b (range: 0 to 0.005)
  intercept: 0,        // computed automatically
  startX: 0            // where correction begins (draggable bar)
}
```

Data Loading & Parsing

Time Value Parsing

The tool intelligently parses time columns that may be in various formats:

- **Numeric**: `123.45` (seconds)
- **Date strings**: `2026-05-11T14:30:00`
- **Time strings**: `14:30:05.123` (HH:MM:SS.ms)

```
function parseTimeValue(value) {
  if (value == null || value === '') return NaN;
  if (typeof value === 'number') return value;

  const str = String(value).trim();
  const numeric = Number(str);
  if (Number.isFinite(numeric)) return numeric;

  // Try ISO date parsing
  const parsedDate = Date.parse(str);
  if (Number.isFinite(parsedDate)) return parsedDate / 1000;

  // Try with T separator
  const parsedNormalized = Date.parse(str.replace(' ', 'T'));
  if (Number.isFinite(parsedNormalized)) return parsedNormalized / 1000;

  // Parse HH:MM:SS format
  const match = str.match(/(\d{1,2}:\d{2}:\d{2}(?:\.\d+)?)/);
  if (!match) return NaN;

  const parts = match[1].split(':');
  const hours = Number(parts[0]);
```

```
  const minutes = Number(parts[1]);
  const seconds = Number(parts[2]);

  return hours * 3600 + minutes * 60 + seconds;
}
```

Normalization

All x-values are normalized to begin at 0 by subtracting the first (base) x-value:

$$x_{\text{normalized}} = x_i - x_1$$

This simplifies time-series analysis and prevents numerical issues with large time offsets.

```
const base = parsedRows[0].x;  // first time value
parsedOriginal = parsedRows.map(point => ({
  x: point.x - base,
  y: point.y
}));
```

Region Statistics

Selection Mechanism

Users define regions by **dragging across the chart**. Each region is bounded by x_0 (start) and x_1 (end).

Region objects store: - `x0`, `x1`: x-axis bounds - `color`: HSLA color for visualization

Statistics Computation

For each region, the tool computes:

1. **Mean**:

$$\bar{y} = \frac{1}{n} \sum_{i=1}^{n} y_i$$

2. **Standard Deviation**:

$$s = \sqrt{\frac{1}{n-1}\sum_{i=1}^{n}(y_i - \bar{y})^2}$$

3. **Point Count**: n = number of data points in region

```
function computeStatsForRegion(region) {
  const points = rawData.filter(
    point => point.x >= region.x0 - 1e-12 &&
             point.x <= region.x1 + 1e-12
  ).map(point => point.y);

  const n = points.length;
  if (n === 0) return { n: 0, mean: NaN, std: NaN };

  const mean = points.reduce((sum, value) => sum + value, 0) / n;

  let std = NaN;
  if (n > 1) {
    const variance = points.reduce(
      (sum, value) => sum + (value - mean) ** 2,
      0
    ) / (n - 1);
    std = Math.sqrt(variance);
  }

  return { n, mean, std };
}
```

Real-time Updates

Statistics are recalculated whenever: - A new region is created - A region is moved or resized - Linear correction is toggled or adjusted - Slider values change

The `updateRegionsTable()` function refreshes the display.

Linear Drift Correction

Physical Motivation

In VLE experiments, sensor signals often drift linearly over time due to thermal effects or instrument drift. This feature removes a linear trend, allowing accurate equilibrium calculations.

Mathematical Model

Given uncorrected signal $y(x)$ where x is time, we model the drift as:

$$\text{drift}(x) = mx + b$$

The corrected signal is:

$$y_{\text{corrected}}(x) = y(x) - (mx + b)$$

where: - m is the **slope** (drift rate per unit time) - b is the **intercept** (y-axis offset)

Constraint: No Discontinuity at Start

To prevent a jump in signal at the correction start point, we enforce:

$$\text{drift}(x_{\text{start}}) = 0$$

This gives:

$$mx_{\text{start}} + b = 0 \implies b = -mx_{\text{start}}$$

Therefore:

$$\text{drift}(x) = m(x - x_{\text{start}})$$

And the corrected signal becomes:

$$y_{\text{corrected}}(x) = y(x) - m(x - x_{\text{start}}) \quad \text{for } x \geq x_{\text{start}}$$

At $x = x_{\textbf{start}}$: correction = 0, so $y_{\text{corrected}} = y$ (unchanged)

For $x > x_{\textbf{start}}$: correction grows linearly

Implementation

```
function computeIntercept() {
  // b = -m * startX ensures drift(startX) = 0
  correction.intercept = -correction.slope * correction.startX;
}

function applyLinearCorrection() {
  if (!parsedOriginal || parsedOriginal.length === 0) return;

  // Start from unmodified data
  rawData = parsedOriginal.map(p => ({ x: p.x, y: p.y }));

  if (!correction.enabled) {
    updateRegionsTable();
    return;
  }

  // Compute intercept to avoid discontinuity
  computeIntercept();
  const m = Number(correction.slope) || 0;
  const b = Number(correction.intercept) || 0;
  const sx = Number(correction.startX) || 0;

  // Apply correction only for x >= startX
  for (let i = 0; i < rawData.length; i++) {
    if (rawData[i].x >= sx - 1e-12) {
      rawData[i].y = rawData[i].y - (m * rawData[i].x + b);
    }
  }

  // Update chart and statistics
  if (chart && chart.data && chart.data.datasets && chart.data.datasets[0]) {
    chart.data.datasets[0].data = rawData.map(p => ({ x: p.x, y: p.y }));
    chart.update();
  }

  updateRegionsTable();
}
```

Interactive Controls

Slope Slider (0 to 0.005, step 0.00001): - Adjust drift rate with high precision - Range is user-customizable via min/max inputs - Changes apply instantly to chart and statistics

Start Position Bar (vertical draggable line): - Click and drag on the chart to reposition - Defines where correction begins - Color: orange (#ffff64cc) for visibility

Range Inputs (min/max): - Customize slider bounds - Useful for focusing on specific drift rates - Step size: 0.0001 for fine control

Reset Button

Restores defaults: - `correction.enabled = false` - `correction.slope = 0` - Slider range: 0 to 0.005 - Recalculates statistics on uncorrected data

---

Chart Visualization

Chart.js Integration

The tool uses **Chart.js 4.3** with custom plugins for visualization:

```
const ds = {
  label: signalLabel,
  data: rawData.map(point => ({ x: point.x, y: point.y })),
  borderColor: '#bfefff',
  backgroundColor: 'rgba(79,195,247,0.15)',
  tension: 0.05,
  pointRadius: 0
};

chart = new Chart(ctx, {
  type: 'line',
  data: { datasets: [ds] },
  options: {
    maintainAspectRatio: false,
    scales: {
      x: { type: 'linear' },
      y: { }
    }
```

```
  },
  plugins: [regionPlugin, verticalBarPlugin]
});
```

Custom Plugins

regionPlugin: Draws region overlays and the startX bar

```
const regionPlugin = {
  id: 'regionPlugin',
  afterDatasetsDraw(chartInstance) {
    const drawCtx = chartInstance.ctx;
    const xScale = chartInstance.scales.x;
    const chartArea = chartInstance.chartArea;

    // Draw regions
    regions.forEach(region => {
      const left = xScale.getPixelForValue(region.x0);
      const right = xScale.getPixelForValue(region.x1);
      drawCtx.fillStyle = region.color;
      drawCtx.fillRect(
        Math.min(left, right),
        chartArea.top,
        Math.abs(right - left),
        chartArea.bottom - chartArea.top
      );
    });

    // Draw startX vertical bar
    if (correction && typeof correction.startX === 'number') {
      const px = xScale.getPixelForValue(correction.startX);
      drawCtx.strokeStyle = 'rgba(255,200,100,0.9)';
      drawCtx.lineWidth = 2;
      drawCtx.beginPath();
      drawCtx.moveTo(px, chartArea.top);
      drawCtx.lineTo(px, chartArea.bottom);
      drawCtx.stroke();

      // Handle for dragging
      drawCtx.fillStyle = 'rgba(255,200,100,0.95)';
      drawCtx.fillRect(px - 4, chartArea.top + 6, 8, 18);
```

```
    }
  }
};
```

Interactive Features

- **Drag to create region**: Click and drag across chart
- **Drag to resize region**: Move region edges (8px threshold)
- **Drag to move region**: Click region center and drag
- **Shift + drag to zoom X-axis**: Resets with "Reset X Zoom" button
- **Drag vertical bar**: Move startX position for correction

Export Functionality

CSV Export

Exports region statistics with headers:

```
region,x_start,x_end,mean_signal,std_signal,n_points
1,15,45,44.932,0.0234,1204
2,98,156,44.891,0.0198,2340
```

```
function generateExportCSV() {
  const sortedRegions = [...regions].sort((a, b) => a.x0 - b.x0);
  const lines = ['region,x_start,x_end,mean_signal,std_signal,n_points'];

  sortedRegions.forEach((region, idx) => {
    const stats = computeStatsForRegion(region);
    const x_start_idx = getIndexForX(region.x0);
    const x_end_idx = getIndexForX(region.x1);

    lines.push([
      idx + 1,
      x_start_idx,
      x_end_idx,
      stats.n >= 1 ? formatSig(stats.mean) : '',
      stats.n > 1 ? formatSig(stats.std) : '',
```

```
      stats.n
    ].join(','));
  });

  return lines.join('\n');
}
```

TSV Copy to Clipboard

Tab-separated format for spreadsheet pasting:

```
region  x_start x_end   mean_signal std_signal  n_points
1   15  45  44.932  0.0234  1204
```

Significant Figures

Values are formatted to user-selected precision (3–8 significant figures):

```
function formatSig(value) {
  if (typeof value !== 'number' || !Number.isFinite(value)) return '-';
  return Number(value).toPrecision(selectedSigFigs);
}
```

Event Flow & State Management

Initialization

1. User loads CSV file
2. `fileInput.addEventListener('change')` triggers
3. PapaParse parses file with dynamic typing
4. `loadParsed()` processes rows:
    - Extracts numeric columns
    - Sorts by x-value
    - Normalizes x to start at 0
    - Populates `parsedOriginal`
5. `applyLinearCorrection()` (no-op if disabled)
6. `createChart()` initializes Chart.js
7. `setupDrag()` attaches mouse handlers

Correction Adjustment

1. User moves **slope slider** → `slopeSlider.addEventListener('input')`
2. Update `correction.slope`
3. Call `computeIntercept()` to ensure no discontinuity
4. Call `applyLinearCorrection()` to rebuild `rawData`
5. Call `updateRegionsTable()` to refresh statistics
6. Chart updates via `chart.update()`

Dragging StartX Bar

1. Mouse down near vertical bar → `draggingStartBar = true`
2. Mouse move: compute new x from pixel position
3. Update `correction.startX`, recalculate intercept
4. Rebuild `rawData` and refresh chart
5. Mouse up: finalize and unlock

Numerical Considerations

Floating-Point Tolerance

Comparisons use epsilon thresholds to handle rounding:

```
// Instead of: point.x === region.x0
// Use:
if (point.x >= region.x0 - 1e-12 &&
    point.x <= region.x1 + 1e-12) { ... }
```

Data Point Indexing

Map continuous x-values to nearest discrete data point:

```
function getIndexForX(xValue) {
  if (!rawData || rawData.length === 0) return 0;

  let closestIdx = 0;
  let closestDist = Math.abs(rawData[0].x - xValue);
```

```
  for (let i = 1; i < rawData.length; i++) {
    const dist = Math.abs(rawData[i].x - xValue);
    if (dist < closestDist) {
      closestDist = dist;
      closestIdx = i;
    }
  }

  return closestIdx;
}
```

Performance Notes

- **Chart recreation**: Avoided during correction updates; uses `chart.update()` instead
- **Region statistics**: Computed on-demand only when displayed
- **Data immutability**: `parsedOriginal` never modified; corrections applied to `rawData` copy
- **Memory**: For typical VLE datasets (~10k points), memory usage is negligible

---

Browser Compatibility

- **Chart.js 4.3**: Modern browsers (Chrome, Firefox, Safari, Edge)
- **File API**: Required for CSV upload (all modern browsers)
- **ES6**: Uses arrow functions, template literals, spread operator
- **Local Execution**: No external API calls; runs entirely in-browser

---

Future Enhancements

- **Higher-order corrections**: Polynomial drift (quadratic, cubic)
- **Multiple corrections**: Independent per-region adjustments
- **Data smoothing**: Moving average or Savitzky-Golay filter
- **Peak detection**: Automatic region identification
- **undo/redo**: Revert recent corrections

- **Export formats**: JSON, HDF5, Parquet

References

- Chart.js Documentation: https://www.chartjs.org/docs/latest/
- PapaParse CSV Parser: https://www.papaparse.com/
- IEEE Standard for Floating-Point Arithmetic (IEEE 754)